Journal of Cloud Computing

RESEARCH Open Access

Online machine learning for proactive threat intelligence and timely DDoS attack prediction

Rrezearta Thaqi[1], Gabriele Lenzini[2], Marcus Völp[2] and Blerim Rexha[1*]

Abstract

Distributed Denial of Service (DDoS) attacks are rapidly escalating in scale and sophistication, posing critical threats to modern cloud infrastructures. Recent reports from Cloudflare show unprecedented attack volumes, 20·5 M attacks in Q1 2025, and sophisticated patterns such as low-rate stealth or bursty bots, underscoring the need for timely, automated threat intelligence. Traditional defenses – static IDS signatures or threshold rules – remain reactive and incur high false-alarm rates. To address these gaps, we propose a real-time ML framework for proactive DDoS defense. We built a unified labeled dataset from public (CIC-IDS2017 DDoS and CIC-DDoS2019) and proprietary, pfSense traffic captures, enriched with MISP threat indicators. In a comparative supervised evaluation, we trained multiple classifiers (SVM, Random Forest, XGBoost) via cross-validated grid search, using accuracy, precision, and recall as key metrics. The architecture routes all traffic through a pfSense gateway into a Wazuh analytics node, with OpenSearch and Grafana, for streaming feature extraction and prediction. Results show all models achieved near-perfect detection (≈99.99% accuracy, 100% precision/recall) on test data. XGBoost slightly outperformed the others and maintained $> 99.9\ \%$ accuracy even under 15,000 events/sec load and was thus selected for deployment. The pipeline sustained inference latency $< 50\,\mathrm{ms}$, enabling timely alerts and automated mitigation. These findings demonstrate that ML-based threat intelligence can greatly enhance DDoS mitigation with minimal false alarms. In addition, we draw inspiration from fault tolerance to protect and improve the availability of the watchmen themselves, by replicating threat intelligence, distributing them geographically, and require them to reach consensus to filter spurious alerts, or worse, compromise of individual instances of the servers on which they execute. Future work will close the loop to automated enforcement, such as dynamic firewall updates, adapt models to evolving attack patterns, and advancing proactive cloud security.

Keywords Cloud computing, DDoS attack, Cloud security, Threat intelligence, Machine learning

*Correspondence:
Blerim Rexha
blerim.rexha@uni-pr.edu
[1]Faculty of Electrical and Computer Engineering, University of Prishtina, Prishtina, Kosovo
[2]Interdisciplinary Centre for Security Reliability and Trust (SnT), University of Luxembourg, Esch-sur-Alzette, Luxembourg

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

Introduction

Distributed Denial of Service (DDoS) attacks have evolved into one of the most potent forms of cyberattacks, capable of overwhelming critical infrastructure by flooding it with illegitimate traffic. The scale and frequency of these attacks are not only persistent but accelerating. Cloudflare's 2025 Q1 DDoS Threat Report [1] revealed 20.5 million blocked DDoS attacks in just three months, representing a 358% year-over-year increase and nearly double the volume from the previous quarter as shown in Fig. 1.

Such evidence highlights the urgency of developing more effective defenses, as adversaries increasingly exploit the elastic and distributed nature of modern cloud services.

In contemporary cloud-based environments, where services are distributed, elastically scaled, and often containerized, the impact of such attacks can be both widespread and difficult to localize [2]. Unlike conventional on-premise systems, cloud-native architectures introduce new complexities in threat surface management, traffic routing, and inter-node visibility. These characteristics call for equally dynamic and adaptive defense mechanisms capable not only of detecting attacks in real time but also of anticipating them based on observed network behavior patterns [3].

Traditional defense strategies often rely on static rule-based intrusion detection systems (IDS), threshold-based firewalls, or manual traffic analysis [4]. While these tools offer a first layer of defense, they are inherently reactive and fall short in environments characterized by rapid workload fluctuations, transient network states, and complex east-west traffic patterns [5]. Moreover, these approaches typically do not scale well in systems composed of ephemeral nodes such as those managed via auto scaling groups in cloud infrastructure. An example traditional setup is illustrated in Fig. 2, where a signature-based IDS inspects traffic against a database of known attack signatures and triggers alerts upon matches.

In contrast to traditional systems that often suffer from high false-positive rates and struggle to adapt to novel attacks, AI and machine learning-driven intrusion prevention systems have demonstrated significantly higher effectiveness. A 2025 comparison study [7] found that AI-based IDS/IPS systems reduced false-positive rates by nearly 30% and achieved much higher accuracy in detecting previously unseen threats while offering scalable, real-time performance where traditional systems frequently failed [7].

Unlike static IDS rules that only match predefined signatures, ML-based systems infer patterns from traffic statistics and contextual features. This data-driven approach enables them to flag variants and previously unseen attacks with greater reliability. Recent work confirms that such models not only sustain high detection accuracy but also cut down on spurious alerts in operational settings [8].

To enhance the detection process, the security community has turned toward threat intelligence (TI), understood here as the use of real-time feeds and contextual indicators—such as IP/domain reputation lists, malicious infrastructure IOCs, and behavioral patterns observed in the wild—to complement local traffic analysis [9]. Combined with ML, these signals move beyond static blacklisting and support predictive analytics that anticipate emerging DDoS activity rather than only reacting once attacks are underway [10, 11].

In parallel, watchmen, which gather threat intelligence, are increasingly also affected by DDoS and other attacks, threatening the availability of alarms, specifically ones of a predictive nature, like ours. To compensate for this, we draw inspiration from the fault and intrusion tolerant systems domain [12] and apply replication and

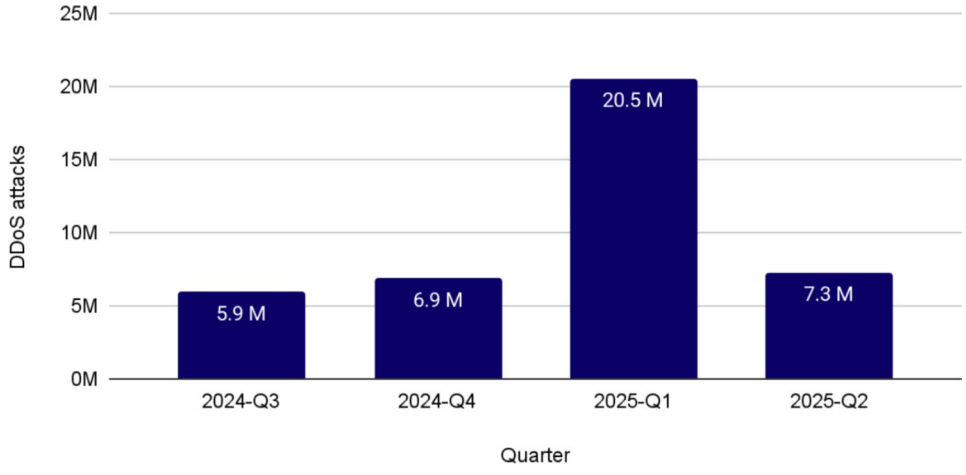


Fig. 1 DDoS attacks by quarter [1]

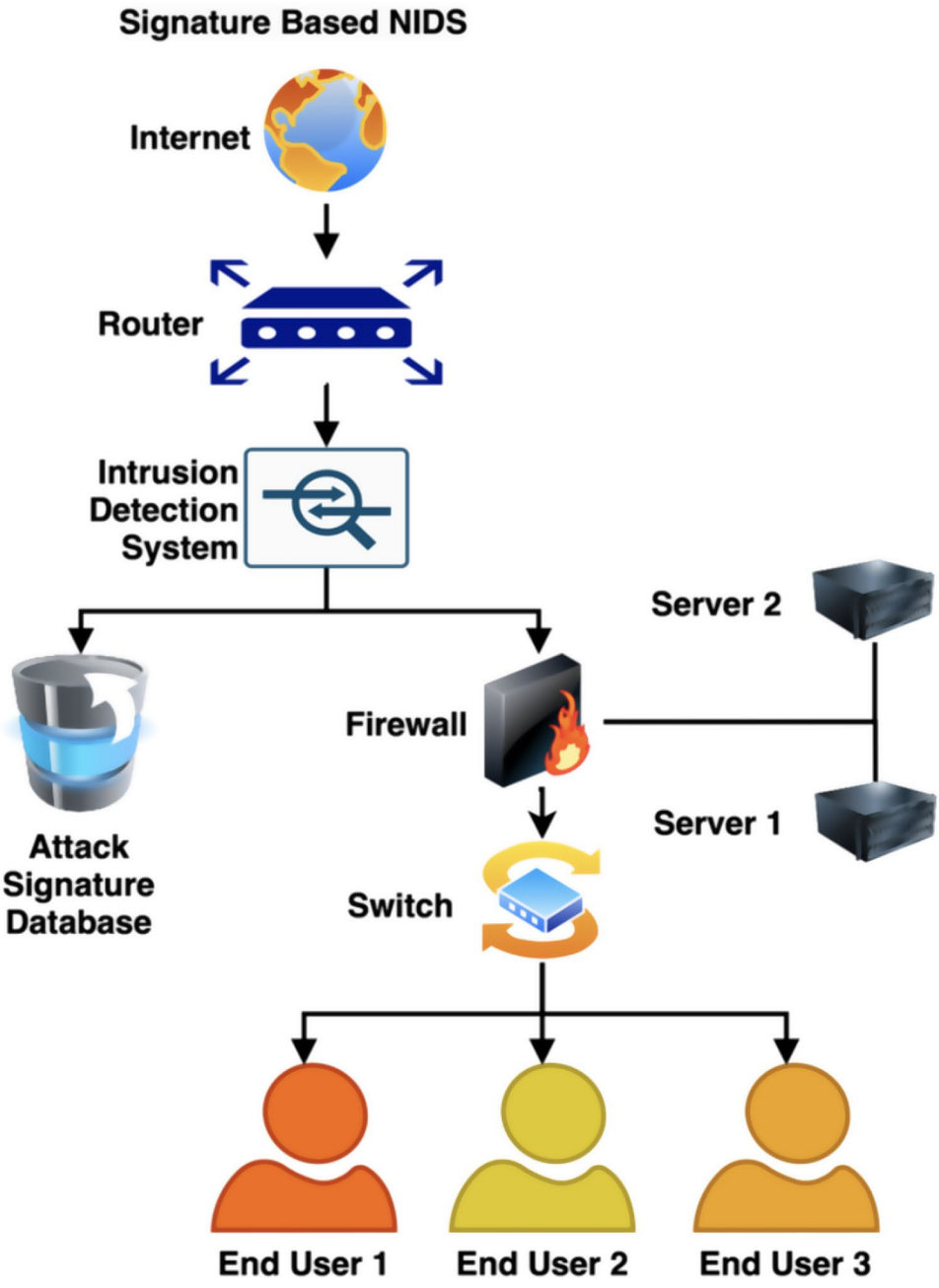


Fig. 2 Traditional signature-based NIDS architecture. Traffic is compared against an attack signature database, and alerts are triggered upon matches. While effective for known threats, such systems are reactive and cannot detect novel or evolving DDoS attacks until new signatures are defined [6]

distributed coordination for maintaining consistency among the decisions and for tolerating regional attacks against our watchmen infrastructure.

Proactive defense mechanisms are essential in the context of DDoS attacks, since reactive measures alone often occur too late to prevent service degradation. Beyond migration and recovery strategies, we integrate prediction-based defenses that forecast attack onset based on traffic flow features. This allows the system to enact preventive actions, such as firewall rule pre-deployment, selective rate limiting, or temporary scaling of replica resources, before the traffic volume reaches saturation. Such proactive techniques directly address the availability threat by reducing the time window in which adversarial traffic can overwhelm the infrastructure.

This paper proposes a novel architecture that combines these paradigms: threat intelligence, machine learning, and distributed coordination. The system leverages pfSense as a traffic monitoring gateway deployed within an AWS Virtual Private Cloud (VPC). All traffic from

a dynamically scaling group of replica nodes is routed through the pfSense instance, enabling full visibility into communication flows. pfSense forwards structured logs to a central analytics node running Wazuh, which integrates with OpenSearch and Grafana for log indexing and visualization.

At the core of the system is a trained machine learning model, deployed on the Wazuh server, that performs DDoS prediction from flow-level features. This enables both real-time detection and early warning. While zero-day threats naturally lie outside the training data, volumetric characteristics like packet/byte rates, SYN/ACK ratios, and source-IP entropy tend to persist across new variants and studies confirm that ML models using these features generalize well to unseen DDoS behaviors [13]. To ensure robustness, we also embed BFT-inspired coordination so replicas reach consensus even during partial failures or compromise.

The contributions of this work are twofold: (1) we introduce a hybrid detection and prediction framework that integrates network observability with machine learning to enhance DDoS mitigation capabilities, and (2) we ensure reliability of detection outcomes and availability of detectors through replication and lightweight consensus among monitoring replicas. While our focus is limited to DDoS attacks where adversaries primarily aim to overwhelm availability rather than compromise nodes, the consensus mechanism helps avoid inconsistent or erroneous alerts from individual detectors. Moreover, it provides coordination among geographically distributed monitoring replicas that are less likely to be affected by the DDoS attack in one region. This system is particularly suited for cloud-native environments where scalability, automation, and proactive threat response are mission-critical.

Machine learning and threat intelligence in the context of geo-replicated cloud security

Cloud computing has transformed modern IT by enabling flexible, scalable, and cost-effective access to computational resources. However, this transition has also amplified the attack surface for cyber threats, particularly DDoS attacks. These attacks attempt to flood a target with overwhelming traffic, depleting critical system resources and causing service outages. As the scale and complexity of cloud systems grow, especially with multi-tenant and distributed architectures, the susceptibility to DDoS increases significantly [14].

DDoS attacks not only affect user accessibility but can degrade inter-service communication, disrupt backend orchestration, and trigger cascading failures in distributed systems. In recent years, attackers have evolved their tactics, including low-rate stealthy attacks, botnet-driven bursts, and application-layer overloads, which evade traditional threshold-based intrusion prevention systems [15].

Machine learning (ML) has emerged as a robust solution to address the limitations of conventional detection systems. Unlike rule-based methods, ML models can learn complex, non-linear patterns in traffic and system telemetry. Classifiers such as Random Forests, Support Vector Machines (SVM), K-Nearest Neighbors (KNN), and Deep Neural Networks have been applied to detect anomalies and malicious behavior in real-time traffic streams [14, 16].

Recent research demonstrated that ML techniques, especially ensemble models and hybrid approaches, can outperform conventional detection systems by achieving higher precision, recall, and lower false-positive rates [17, 18]. Studies also highlight the need for adaptive learning algorithms capable of managing the challenges posed by zero-day attacks and the concept drift associated with evolving threats [19].

Threat intelligence (TI) augments ML by introducing contextual awareness—fusing external indicators of compromise (IOCs), IP reputation scores, dark web surveillance, and vulnerability feeds into the decision-making process. This integration enhances ML classifiers by weighting features based on threat context, thereby improving detection accuracy and enabling proactive mitigation strategies [18].

Moreover, threat intelligence aids in reducing the latency of response by facilitating real-time updates on emerging threats and known malicious infrastructure. For example, fusing dynamic threat feeds with network flow analysis enables rapid detection of known botnet signatures and command-and-control (C2) traffic, even if payload-level inspection is encrypted or obfuscated.

Complementary to these findings, Benjamin [20] explored the fusion of real-time threat intelligence feeds with machine learning pipelines to enhance proactive defense in dynamic network environments. Although not tailored to BFT-based systems, this work highlights the importance of integrating external threat indicators into ML workflows, supporting our argument that threat-intelligence-driven prediction strengthens resilience when combined with lightweight consensus mechanisms.

While DDoS primarily targets availability through network exhaustion rather than direct node compromise, maintaining consistent and reliable detection across replicas remains critical. For this reason, we adopt lightweight consensus mechanisms inspired by solutions from the fault and intrusion tolerant systems domain.

To achieve resilience, many cloud-based distributed systems, specifically blockchains, databases, and real-time services [21], adopt Byzantine Fault Tolerant (BFT) protocols, such as PBFT (Practical Byzantine Fault Tolerance) [22] or GroupBFT [23]. These protocols ensure

system consistency even when a subset of nodes behave maliciously or arbitrarily. In a *3f+1* configuration, BFT ensures service availability as long as fewer than f replicas are faulty. Crash fault tolerance (e.g., as in Paxos [?]), are optimized for less potent adversaries that can present replicas only from responding, but not alter their messages.

While in this work, we do not consider compromise of monitoring replicas, we find a similar situation where due to the nature that observations are not symmetric by all replicas, we have to cope with disagreements in the assessment what individual replicas observe and with false alarms due to how they are connected to the system they observe.

It remains so an interesting topic of future work to also evaluate ML-based DDoS detection and threat intelligence for replicated systems, such as blockchains or systems deploying various forms of BFT protocols, even so currently available datasets are rare, hampering reproducability and benchmarking, requiring synthetic loads to be used, like CICIDS-2017, CIDDS-001, or NSL-KDD for cloud DDoS contexts [17].

The integration of machine learning, enriched with threat intelligence, and tailored to the operational intricacies of BFT protocols offers a promising research direction. Such systems could monitor consensus health, replica communication logs, and network metrics in feeding this multidimensional data into hybrid ML architectures. Real-time predictions could trigger defensive strategies such as rate limiting, replica redistribution, or dynamic quorum adjustment before service-level objectives are violated.

This novel intersection of intelligent detection and protocol-specific resilience represents a critical step forward in securing next-generation cloud infrastructures.

Related work

The intersection of machine learning (ML) and threat intelligence (TI) in the context of DDoS detection in cloud environments, especially those incorporating Byzantine Fault Tolerance (BFT) is an evolving field. Recent studies provide critical insights into ML-based intrusion detection systems (IDS), emphasizing model accuracy, adaptability, and scalability.

Sumukh and Sandhya [24] evaluated traditional ML algorithms including Random Forests and Decision Trees alongside deep learning (DL) models like Artificial Neural Networks (ANN) and Long Short-Term Memory (LSTM). Their work highlighted LSTM's superior adaptability for time-dependent traffic anomalies. Similarly, Subrmanian et al. [25] used the CSE-CIC-IDS2018 dataset to address the problem of identifying DDoS attacks within a cloud environment by multiple logistic regression analysis employments.

Makkawi and Yousif [26] conducted a systematic review of ML-based DDoS defenses in cloud environments, concluding that ensemble approaches consistently outperformed single-model classifiers due to their robustness to data imbalance and attack variability. These findings are echoed by Mishra et al. [27], who implemented a comparative framework using KNN, Random Forest, and Naive Bayes, with Random Forest achieving 99.76% accuracy.

Beyond these conventional and ensemble approaches, recent work has explored a hybrid pipeline in which a Probability of Fitness–based Billiards-Inspired Optimization (PF-BIO) algorithm first selects features through global search, Fisher discriminant analysis, and feature–class correlation, then optimizes fusion weights across those subsets. This achieved a detection accuracy of 97.05%, highlighting the promise of combining metaheuristic feature selection with temporal deep learning for precise DDoS identification [28].

To address low-rate and stealthy DDoS attacks, Manimaran et al. [29] proposed an adaptive framework using Decision Trees, specifically optimized for container-based cloud architectures. Their design emphasizes real-time response and high interpretability, making it suitable for cloud-native deployments. Likewise, AlSaleh et al. [30] introduced the Bayesian Convolutional Neural Network (BaysCNN) model, enhanced through feature fusion, achieving 99.79% accuracy across multiple DDoS classes using the CICDDoS2019 dataset.

Green [31] offered a broad view on integrating ML within cloud security, advocating for hybrid frameworks that blend ML-based detection with traditional IDS systems. This multi-layered defense approach aligns well with the recommendations by Corrêa et al. [32], who used OpenStack-native telemetry in a real-world testbed. They demonstrated that kNN and Random Forest could achieve up to 87% accuracy in distinguishing between SYN_Flood and GET_Flood attacks, outperforming Snort in certain cases.

Incorporating threat intelligence (TI) into machine learning-based detection pipelines has emerged as a powerful enhancement in the field of network security. As highlighted in the literature review by [3], the fusion of TI improves the contextual understanding of network events, thereby increasing the accuracy and responsiveness of DDoS detection systems.

Krishna et al. [33] expanded on this by proposing a cloud security framework using CNNs and RNNs, supplemented with ensemble strategies. They noted significant accuracy improvements through feature engineering driven by threat intelligence feeds.

Suneetha Bulla and S.K. [17] provided a dataset-focused review, comparing results across popular benchmarks like NSL-KDD, CIDDS-001, and CICIDS2017.

Their synthesis revealed that data quality and attack diversity greatly influence ML model performance. This aligns with Corrêa et al. [32], who found that native telemetry and macroscopic monitoring offered more practical signals than raw packet inspection alone.

Meanwhile, Green [31] and Chauhan [34] emphasized the importance of real-time inference and minimal latency in deployment, a crucial consideration for systems with BFT protocols where delays could trigger consensus failure.

Recent efforts have further advanced DDoS detection accuracy. Abiramasundari and Ramaswamy [35] proposed a PCA-enhanced feature selection approach (EDAD) combined with supervised classifiers such as Random Forest and SVM on CIC-IDS and CICDDoS datasets, achieving detection rates close to 99%. Their study reinforces the effectiveness of ensemble-based methods for handling complex and high-dimensional DDoS traffic patterns in cloud environments.

Despite significant progress, none of the surveyed studies specifically tailor their approaches for BFT-based replicated systems or machine learning combination with threat intelligence. While Corrêa et al. [32] mention distributed systems and telemetry, they stop short of addressing protocol-layer variables like quorum latency or replica message entropy. This gap highlights the need for integrating BFT state data, such as round durations, view changes, and replica failure patterns, into ML feature sets.

Recent contributions further emphasize the relevance of BFT for securing cloud-native infrastructures. Feng et al. [36] introduced a hash-based multi-valued validated asynchronous BFT protocol that reduces consensus latency and communication overhead, improving practicality in adversarial environments.

Wu et al. [37] proposed BFTBrain, an adaptive consensus framework that employs reinforcement learning to dynamically switch between protocols under fluctuating workloads and fault conditions, achieving significant performance gains in cloud deployments.

Similarly, Fu et al. [38] presented Hamster, a synchronous BFT protocol that leverages erasure coding to achieve near-linear throughput scalability while tolerating up to 50% malicious replicas.

Although these works advance the state of the art in consensus, they primarily optimize protocol performance and adaptability, without integrating threat intelligence or machine learning into the detection loop. Our work addresses this gap by embedding lightweight BFT-inspired coordination within a proactive ML-driven defense pipeline for DDoS prediction in cloud environments.

Problem statement and threat model

Cloud-native services running on elastic, auto-scaling infrastructures face an increasingly sophisticated threat from DDoS attacks that can evolve in both volume and vector patterns within seconds. In such environments, traditional perimeter-based defenses, whether deployed as static load balancers or signature-driven intrusion detection systems struggle to maintain visibility across rapidly provisioned and deprovisioned instances. As replica counts fluctuate, so too do east-west traffic volumes, often reaching hundreds of thousands of packets per second. Any delay in detecting anomalous flows, even on the order of a few seconds, can allow attack traffic to saturate network interfaces or consume critical CPU cycles, causing service degradation or complete downtime before a reactive threshold is ever reached [39].

Moreover, conventional defense postures remain fundamentally reactive. Static thresholds configured in rule-based IDS systems fail to anticipate subtle, low-rate assaults that evade detection by staying below fixed limits, and they generate false alarms during legitimate traffic surges such as flash crowds. Without a predictive mechanism to forecast impending volumetric spikes based on evolving flow statistics, metrics like packets per second, bytes per second, SYN/ACK ratios, and source-IP entropy—operators are left responding to attacks only after they have begun. In highly dynamic cloud environments, this "see-then-respond" approach is too slow to prevent resource exhaustion and its cascading effects on dependent services.

Compounding the challenge is the need to consolidate and process vast quantities of log data in real time. pfSense gateways, when deployed as traffic inspection points, can forward syslog or flow records from multiple replicas at line rate, resulting in tens of gigabytes of logs per hour. Extracting sliding-window features for machine-learning inference—calculating per-second aggregate counters and entropy measures—demands an efficient, in-memory pipeline that can keep pace with incoming streams. Batch-oriented workflows, such as writing CSV snapshots for offline analysis, are unsuitable; feature engineering and classification must occur continuously without dropping packets or overwhelming host resources.

Reliability of detection outcomes is equally critical. In a distributed monitoring architecture, a single compromised or overloaded node can undermine the integrity of the entire detection fabric. Ensuring that all replicas reach consensus on whether a DDoS event is unfolding requires a fault-tolerant coordination layer. Drawing on principles from Byzantine Fault Tolerance (BFT), our framework must maintain agreement across a cluster of monitoring nodes even when up to f out of $3f+1$ replicas behave arbitrarily, whether due to software faults,

network partitions, or malicious compromise. Absent such mechanisms, fragmented or inconsistent alerts will lead to ineffective or conflicting mitigation actions.

A practical DDoS detection framework should plug right into the tools we already use rather than forcing you to rip and replace your infrastructure. For example, you could lean on pfSense for gateway monitoring, Wazuh for SIEM services, OpenSearch for log indexing, and Grafana for alert dashboards. Ideally, the system would tap into pfSense's syslog feed, run lightweight feature extraction and ML inference on your Wazuh agents in under a second, and then ship neatly formatted alerts into your OpenSearch indices—all without demanding a heavy, custom rollout. By building on these familiar components, you get a real-time, proactive defense that scales with your cloud environment and helps you spot and stop DDoS attacks before they take hold.

The ability to predict an impending DDoS attack even a few seconds or minutes in advance enables the system to initiate preventative actions—such as rerouting traffic, instantiating additional defense resources, or live-migrating target services—before the attack peaks. By ensuring that countermeasures are already in place when the attack volume surges, early warning significantly reduces downtime and performance degradation.

Methodology

The proposed system is designed to detect and predict DDoS attacks by integrating real-time traffic monitoring, log analysis, and machine learning–based prediction within a cloud-native infrastructure. In order to achieve both high accuracy and low latency, our methodology proceeds in two major phases: (1) training and model selection phase, and (2) deployment and inference phase.

Data collection and model selection

We begin by constructing a comprehensive, unified dataset that combines multiple sources of labeled traffic. First, we ingest the publicly available CIC-IDS2017 DDoS and CIC-DDoS2019 capture, extracting per-flow statistics such as packets per second (PPS), bytes per second (BPS), TCP SYN/ACK ratios, etc. Next, we merge these records with our proprietary pfSense–derived traffic logs that were gathered during a certain period of time as real-time logs are continuously collected from pfSense, new traffic samples are appended to the dataset for analysis. We emphasize that the proposed system operates in an online manner with continuous data ingestion and real-time inference. From the pfSense logs we could get 15 features including source ip, protocol, packet length, etc. Based on the traffic, we get at least 5 pfSense logs per millisecond. Ground-truth labels for pfSense traffic were generated automatically by fusing Suricata signature alerts with Wazuh correlation rules, requiring independent confirmation from both systems to assign a malicious label. Finally, we enrich every flow with threat-intelligence indicators drawn from MISP feeds: source and destination IPs, domains, and network ranges tagged as malicious are encoded as binary IOC flags and scaled risk scores. The unified dataset contains more than 2,000,000 rows.

Table 1 summarizes the data distribution in datasets including benign vs. attack class distribution:

Table 1 Data size in datasets and benign vs. attack class distribution

Source	Full Dataset	
	Benign	**DDoS**
CIC-IDS2017	2,702,716	128,027
CIC-DDoS2019	~56,863	~2,137,764
pfSense logs	All collected	All collected
Total	**~2.76 M**	**~2.2 M**

For testing purposes, we did replay-based oversampling for pfSense attacks to increase the number of malicious samples in the dataset.

Because the source datasets contained different feature schemas, we performed a harmonization step that maps all flows to a unified feature space. Only features available consistently across all sources were retained, and missing values from dataset-specific fields were imputed or dropped as appropriate. This resulted in a single standardized representation that enables joint training and evaluation.

We applied temporal partitioning to prevent leakage between training and evaluation sets. All CIC-IDS2017 and CIC-DDoS2019 traffic, along with early benign pfSense captures and MISP feeds, formed the training set. All later pfSense traffic and simulated attacks served exclusively as the test set. Flows from the same connections or replayed attack traces appeared in only one set.

On this enriched corpus, we evaluate three candidate classifiers: SVM, Random Forest and XGBoost, each configured to ingest the same feature vector of flow statistics plus IOC enrichments together with the deep learning algorithm and three AI models: OpenAI, DeepSeek and Grok, which were exploratory experiments. Hyperparameter optimization is performed via grid search on the validation split, and models are compared according to precision, recall, and F1-score. Given the operational priority of minimizing false alarms in production, we select the model achieving the highest precision as our production predictor.

System architecture and data pipeline

At runtime, our architecture comprises four cooperating layers:

Traffic monitoring layer

A pfSense instance, deployed on an Amazon EC2 within a VPC, functions as the network gateway for an Auto Scaling Group of replica EC2 instances. All ingress and egress traffic from these replicas is routed through pfSense, providing a single point of packet capture and flow logging. Suricata is enabled on pfSense to augment raw flow records with deep-packet alerts.

Log aggregation and preprocessing layer

pfSense forwards its syslog records in real time over UDP to a central monitoring node running Wazuh. An rsyslog listener captures this stream into a local file (/var/log/pfsense.log), and a custom parser ingests each new line to extract the same feature set employed during training: PPS, BPS, SYN/ACK ratios, entropy measures, and IOC flags. All parsing and feature computation occur in memory in a sliding-window fashion to maintain sub-second throughput. The logs transferred to the Wazuh server are the logs that pfSense monitor from the replicas.

Machine learning prediction layer

A continuously running daemon on the Wazuh host loads the preselected model and monitors the parsed log buffer. For each assembled feature vector, it executes model.predict and to produce a binary decision (attack vs. benign) and a confidence score. When the confidence exceeds a predefined threshold (e.g., 0.90), the daemon immediately emits a JSON-formatted alert—including timestamp, replica identifier, confidence, and key metrics—into Wazuh's active-responses log.

Alerting and visualization layer

The JSON alerts trigger custom detection rules in Wazuh, which index them into OpenSearch under wazuh-alerts-*. Grafana dashboards are then configured to display real-time panels of prediction frequency, confidence heatmaps, per-replica traffic trends, and alert timelines, enabling security operators to visualize and investigate ongoing threats instantly.

Distributed resilience strategy

To guard against failures or malicious behavior within the replica cluster, our framework incorporates principles inspired by Byzantine Fault Tolerance (BFT). Instead of relying on a full Practical Byzantine Fault Tolerance (PBFT) protocol—whose communication overhead is unnecessary for a pure DDoS-threat context—we adopt a lightweight consensus scheme that validates high-confidence alerts before mitigation. Each replica independently processes incoming flow telemetry and emits a binary prediction verdict. A mitigation action (e.g., dynamic firewall rules via pfSense's API) is triggered only if a quorum of replicas agree, ensuring that spurious alerts from an individual node do not result in inconsistent or premature responses. Figure 3 illustrates the architecture of our framework.

Beyond consensus, replicas periodically checkpoint their state (e.g., sliding window statistics and detection outcomes) to ensure uninterrupted operation under failover. By deploying replicas across separate availability zones, the framework preserves monitoring continuity even when a region is degraded or isolated by a volumetric attack. Replica nodes resynchronize upon recovery, preventing detection reset and improving robustness against both infrastructure outages and targeted saturation.

To validate resilience against adversarial behavior, we model Byzantine failure modes within the quorum logic. In a standard $3f + 1$ configuration with $N = 4$ replicas and $f = 1$ malicious node, the compromised replica executed two independent strategies: (i) injecting spurious alerts in an attempt to enforce false mitigations, and (ii) suppressing legitimate alerts to delay response. We evaluated consensus failures, false-positive activations, and mitigation latency under these adversarial scenarios.

To simulate malicious or inconsistent replicas, we modeled Byzantine behavior analogous to model poisoning in federated learning systems, where corrupted nodes intentionally deviate from consensus updates. This adversarial model aligns with prior work such as Li et al.'s [40] which provides a foundation for simulating integrity-compromised participants within distributed environments.

We define our threat model under authenticated asynchronous communication and assume up to $f = \lfloor \frac{N-1}{3} \rfloor$ replicas may behave arbitrarily. A mitigation decision is enacted only when a quorum of $Q = 2f + 1$ matching alerts is reached; otherwise, the system enters a safe-halt mode with default deny policies until quorum is restored. Missing votes exceeding a timeout interval are treated as non-confirmations. This ensures immunity to ties, suppressed alerts, or message withholding.

We additionally simulated scenarios with $f = 2$ faulty replicas in a cluster of $N = 7$ replicas ($Q = 5$). As expected

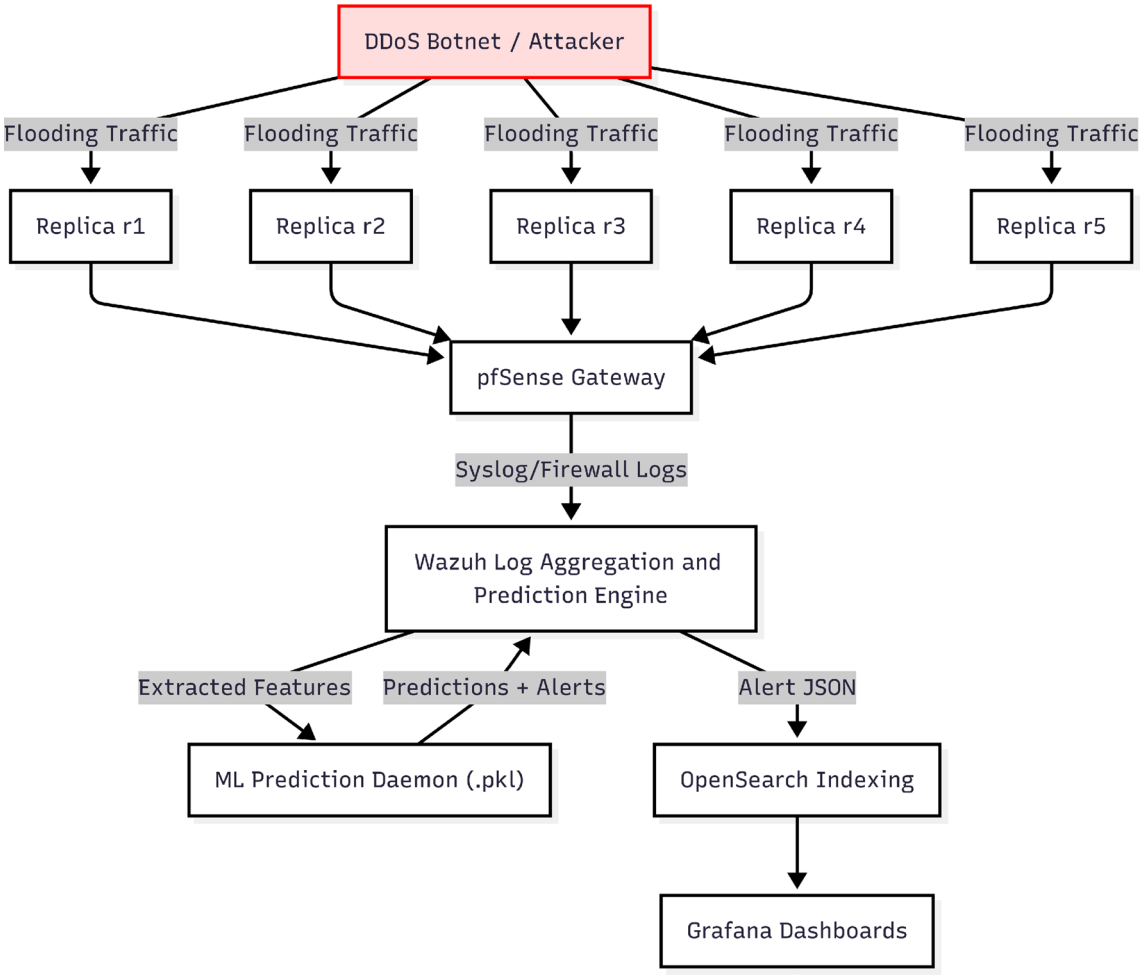


Fig. 3 Architecture of the proposed framework

Table 2 Consensus outcomes under varying Byzantine failures

N	f tested	Quorum Achieved?	False Mitigation?
4	0–1	Yes	No
7	2	No (safe-halt)	No

under the $3f + 1$ bound, mitigation was blocked rather than incorrectly triggered, validating that safety is preserved even under multiple Byzantine failures. Table 2 summarizes system behavior across fault configurations.

Implementation

Model training

Multiple labeled datasets were consolidated, including the CIC-IDS2017 DDoS CSV, CIC-DDoS2019 CSV and proprietary pfSense logs with simulated attack and benign traffic. Raw data columns were first cleansed: categorical fields were label-encoded, timestamp columns dropped, and any NaN or infinite values imputed. Only features available consistently across all sources were retained at the time of datasets merging. The

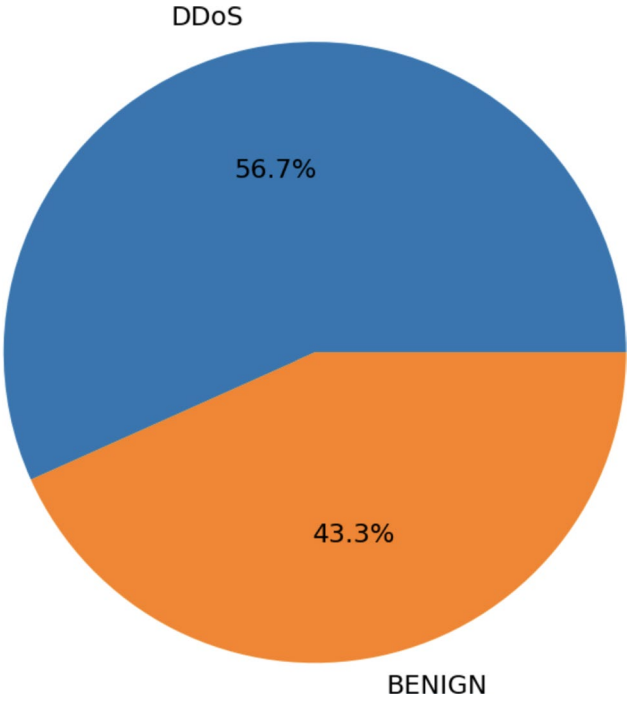


Fig. 4 Distribution of DDoS and BENIGN traffic samples in the consolidated dataset

resulting features matched the flow-level schema used for model training. The CIC-IDS2017 dataset had 85 flow-level features and the CIC-DDoS2019 dataset had 88 features. After harmonization across all datasets (public + pfSense), we retained 15 common features.

To better understand the foundation of our experiments, we first examine the composition of the unified dataset. This provides insight into the balance between attack and benign samples, which directly affects model training and evaluation. Figure 4 illustrates the resulting distribution of DDoS versus BENIGN traffic in the consolidated dataset.

From the processed dataset, we extracted statistical and protocol-based features such as Flow Packets/sec, Flow Bytes/sec, SYN and FIN Flag Counts, packet length statistics (mean, std), inter-arrival time metrics, and binary threat-intelligence flags from MISP. Feature selection initially employed SelectKBest with ANOVA F-test to efficiently filter statistically relevant predictors. To validate that the selected features are not dependent on linear assumptions alone, we compared this ranking against the embedded feature importances learned by a Random Forest classifier trained on the same input vectors as also shown in Listing 1:

Listing 1 Feature selection and ranking code

```
selector = SelectKBest(score_func=f_classif, k=10)
selector.fit(self.X_train, self.y_train)
selected_features = feature_names[selector.get_support()]

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(self.X_train, self.y_train)

rf_importances = rf.feature_importances_
```

The two rankings were then intersected to quantify stability of selection. The top 10 features formed the input vector for model training as it is shown in Fig. 5.

Three candidate classifiers: Support Vector Machine, Random Forest, and XGBoost were trained using 5-fold cross-validated grid search over hyperparameter grids (e.g., C and kernel for SVM; n_estimators and max_depth for tree models; learning_rate and n_estimators for XGBoost). A part of the method implemented to train the classifiers is shown in Listing 2.

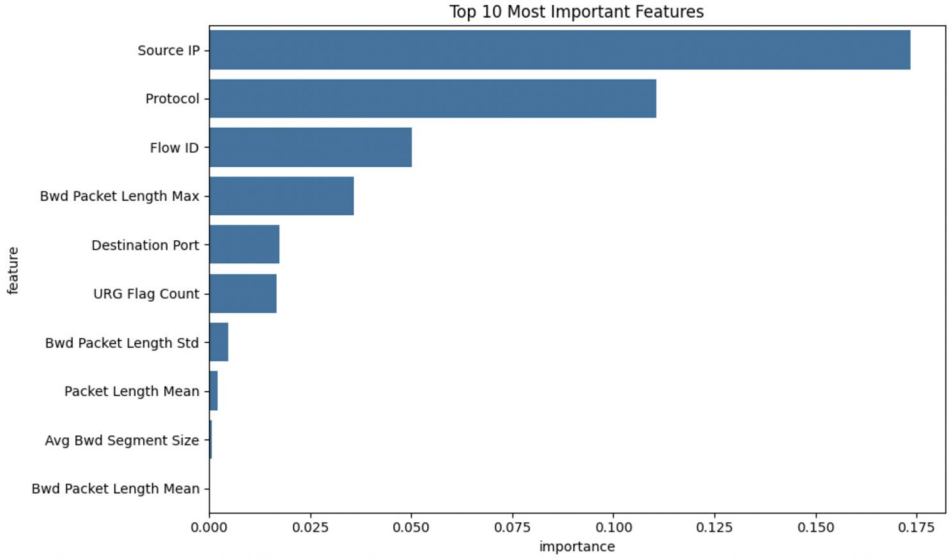


Fig. 5 Feature importance ranking

Listing 2 A part of the method used to train the classifiers

```
def train_ml_models(self):
    models = {
        'SVM': {
            'model': SVC(),
            'params': {
                'C': [0.1, 1, 10],
                'kernel': ['linear', 'rbf']
            }
        },
        'Random-Forest': {
        'model': RandomForestClassifier(),
        'params': {
            'n_estimators': [50, 100, 200],
            'max_depth': [None, 10, 20, 30]
        }
    },
    'XGBoost': {
        'model': XGBClassifier(),
        'params': {
            'learning_rate': [0.01, 0.1, 0.5],
            'n_estimators': [50, 100, 200]
        }
    }
}
```

To find the best algorithm among the three of them, their models were evaluated on an 80/20 train–test split, optimizing for F1-score, and precision was prioritized to reduce false alarms. Listing 3 below shows the process of finding the best algorithm:

Listing 3 Part of the code to find the best algorithm

```
for model_name, model_results in ml_results.items():
    accuracy = float(model_results['accuracy'])
    best_params = model_results['best_params']
    confusion = model_results['confusion_matrix']
    results_list.append({
        'Model': model_name,
        'Accuracy': accuracy,
        'Best Parameters': best_params,
        'Confusion Matrix': confusion
    })
    if accuracy > best_accuracy:
        best_accuracy = accuracy
        best_model_name = model_name
```

As for the deep learning baseline, a fully connected neural network with two hidden layers (64 and 32 units, ReLU activation, BatchNormalization, and Dropout 0.3) was implemented in Keras. The network was trained using Adam optimization, early stopping (patience = 10), and learning rate reduction on plateau. Training converged within 50 epochs, achieving 99.9203% test accuracy.

From our experiments we got a precision and recall score of 1.0 for all the algorithms, so the best algorithm was chosen based on the value of the accuracy.

Although our core contribution focuses on structured-flow machine learning models optimized for real-time inference, we additionally conducted exploratory experiments with large language models (LLMs) to assess their suitability for classifying flow-level network telemetry. These experiments were not used for model selection or deployment decisions. Full experimental details are reported in Appendix A.

Cloud infrastructure provisioning and orchestration

The cloud environment was deployed on AWS using standardized templates that provisioned separate monitoring and replica domains. pfSense, Wazuh, OpenSearch, and Grafana were installed via post-deployment scripts executed through AWS Systems Manager to ensure consistent, hardened configurations. Figure 6 illustrates the deployed EC2 instances in AWS, including the pfSense gateway, Wazuh analytics node, and replica servers.

Virtualized firewall deployment and log capture

In our implementation, pfSense v2.6 was deployed as a dual-homed virtual appliance within the Monitoring Subnet, leveraging one interface for WAN connectivity and another for LAN connectivity to the Replica Subnet. This placement enables pfSense to serve as the single choke point for all east–west and north–south traffic, ensuring comprehensive visibility and control.

To enhance detection fidelity, Suricata v6.0 (Emerging Threats Open ruleset) was installed on the pfSense appliance. Suricata performed deep-packet inspection of mirrored replica traffic, generating EVE-JSON alerts for signatures indicative of volumetric or protocol-based anomalies.

Both pfSense filter logs and Suricata EVE-JSON output were directed to the local syslog daemon. In pfSense's GUI, Remote Syslog forwarding on UDP 514 was enabled, targeting the Wazuh analytics node. Facilities System, Firewall, and local0 were selected for transmission, ensuring that all pertinent logs arrived in real time for downstream SIEM processing. Figures 7 and 8 illustrate the pfSense rsyslog forwarding configuration and the Suricata logging settings.

Centralized log aggregation and security analytics

Before pfSense starts to monitor the logs, some additional configurations are put in place. What pfSense does in this framework, is to forward all the logs needed to the Wazuh server. The log forwarding was done through

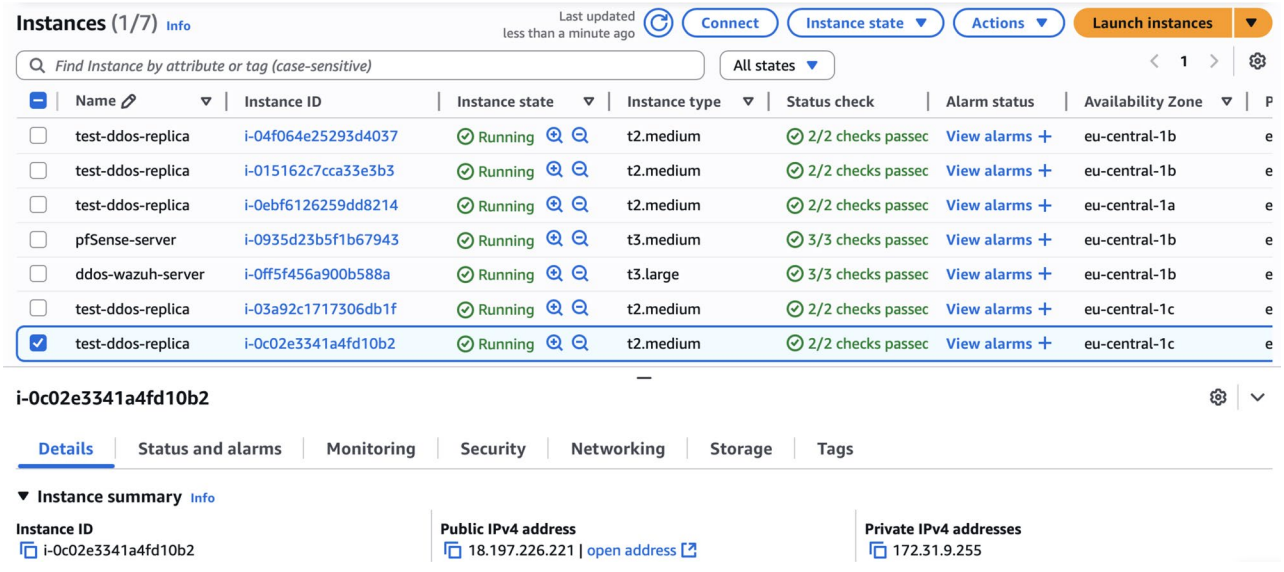


Fig. 6 Deployed servers in AWS

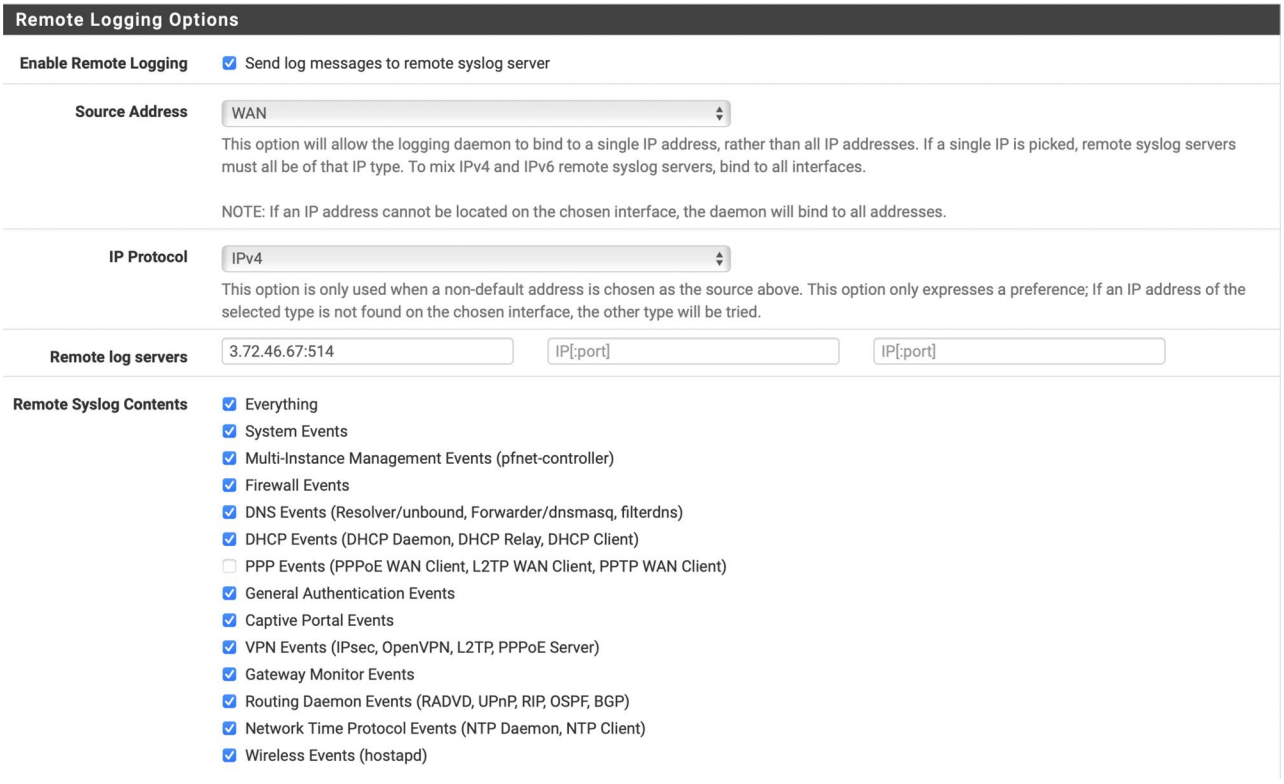


Fig. 7 pfSense config for rsyslog forwarding

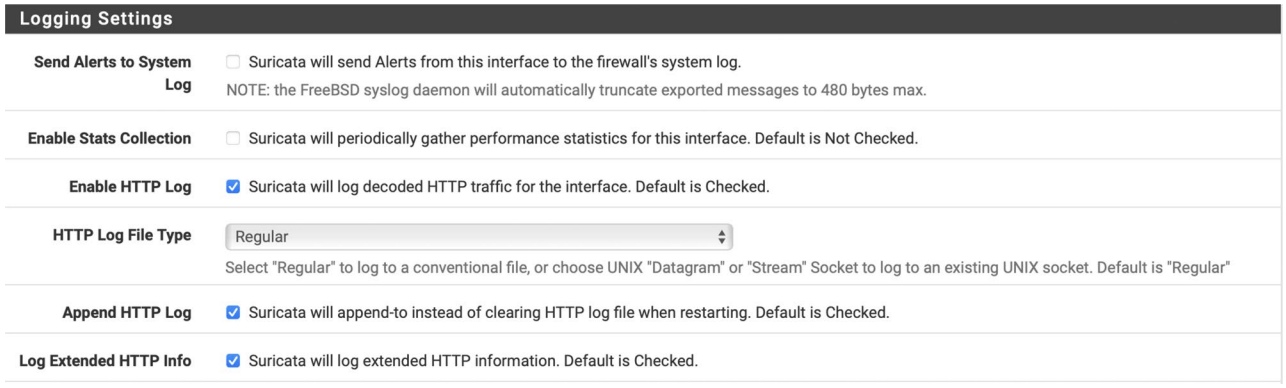


Fig. 8 Suricata logging settings

rsyslog on port 514. In order to be able to do this, on the Wazuh server we create the file sudo /etc/rsyslog.d/10-pfsense.conf and put the configuration as in Listing 4.

Listing 4 Rsyslog configuration in Wazuh node

```
module(load="imudp")
input(type="imudp" port="514")

# Log pfSense traffic to its own file
if $fromhost-ip == '<pfSense-IP>' then {
    action(type="omfile" file="/var/log/pfsense.log")
    stop
}
```

This allows all the pfSense logs to be forwarded to Wazuh after adding also the configuration at the pfSense side.

As pfSense's role is to monitor the traffic of the replicas, on each of the replicas, there was a configuration applied in their user data for all their logs to be forwarded to pfSense as in Listing 5:

Listing 5 Logs forwarding from replicas to pfSense

```
cat <<EOF | sudo tee /etc/rsyslog.d/50-replica-logs.conf
*.* @<pfSense-IP>:514
```

With this configuration all the traffic logs of the replicas are monitored by pfSense and then they are forwarded to the Wazuh server to continue the prediction process for the DDoS attacks.

Real-time detection service

Following the establishment of centralized logging and analytics, the real-time detection mechanism represents the critical component of our defensive infrastructure against DDoS attacks.

The real-time detection service is powered by the previously trained XGBoost model, selected due to its superior precision and performance metrics demonstrated during rigorous model evaluation phases. This predictive engine operates by continuously ingesting log data collected from the pfSense gateway and Suricata

deep-packet inspection systems. Logs arriving in real-time from the replicas to pfSense and subsequently forwarded to the Wazuh node are parsed and subjected to the exact preprocessing pipeline utilized during model training. Specifically, incoming data undergoes label encoding for categorical fields, normalization of numerical features, and immediate statistical aggregation to match the feature set the model expects.

The prediction pipeline is implemented as an asynchronous Python-based inference daemon hosted on the Wazuh analytics server. This daemon utilizes an event-driven architecture to ensure low-latency processing. Key features are extracted from the aggregated logs, including packet and byte throughput rates, SYN and FIN TCP flag distributions, packet length statistics, and binary indicators derived from MISP threat intelligence indicators. The resulting feature vector is passed to the XGBoost model, producing a probability-based prediction regarding the presence of a potential DDoS attack.

Operationally, the service implements a configurable sliding-window mechanism of 60 seconds, aggregating and recalculating metrics of the model used for prediction continuously to maintain an accurate representation of the real-time network conditions. This approach balances the trade-off between rapid detection capability and sufficient statistical accuracy to avoid false positives.

To enhance accuracy and responsiveness, the prediction service integrates a confidence threshold mechanism. If also the set of replicas agrees on a DDoS prediction based on the predefined threshold (in our case, 0.9), actionable alerts are triggered. These alerts are structured as JSON-formatted messages containing critical contextual information, including confidence scores, feature values at the moment of detection, and timestamps. Generated alerts are immediately written to Wazuh's active-responses log, leveraging Wazuh's powerful correlation rules and active response capabilities to enact automated security countermeasures rapidly. All the traffic logs labeled as DDoS attacks or not, are saved to the data being used to increase the accuracy of prediction for future logs. The snippet shown in Listing 6 demonstrates how the system aggregates feature vectors over a fixed 60-second sliding window, an established method in real-time anomaly detection, then executes XGBoost model (chosen for its accuracy and speed in DDoS detection tasks), applies a high-confidence threshold of 0.9, and finally formats structured alerts suitable for Wazuh.

Listing 6 Snippet of the prediction process

```
if len(window) < SLIDING_WINDOW_SIZE:
    return

X = np.mean(window, axis=0).reshape(1, -1)
proba = float(model.predict_proba(X)[0, 1])

if proba > CONF_THRESHOLD and check_replicas_agree(proba):
    alert = {
        "timestamp": datetime.utcnow().isoformat(),
        "confidence": round(proba, 4),
        "features": X.tolist(),
    }
    write_alert(alert)
    print(f"DDoS detected | Confidence: {proba:.4f}")
else:
    print(f"No attack detected | Confidence: {proba:.4f}")
```

Alert data from the Wazuh analytics platform is subsequently indexed into OpenSearch, providing robust search and retrieval capabilities. Grafana dashboards are dynamically updated based on these indexed alerts, offering operators an intuitive visualization interface for real-time network status, attack predictions, confidence score distributions, and detailed per-replica analytics. Dashboards include real-time gauges indicating prediction confidence levels, time series graphs showing alert frequencies, and heat maps identifying specific replicas experiencing anomalous traffic conditions. Beyond visualization, alerts also feed into notification channels such as AWS SNS to notify the users of the system.

If one region or replica is degraded or taken offline completely by a DDoS attack, unaffected replicas seamlessly maintain detection and response continuity.

Table 3 Algorithm testing results for accuracy, precision and recall for the dataset used

	Accuracy	Precision	Recall
SVM	0.999956	1.0	1.0
Random Forest	0.999978	1.0	1.0
XGBoost	0.99999	1.0	1.0
Deep Learning	0.99887	1.0	1.0

To address evolving DDoS tactics, the framework continuously collects new pfSense traffic samples along with their consensus-validated labels. These samples are appended to the training repository, and a periodic retraining job refreshes the model parameters using the latest data snapshot. This adaptive learning loop enables the system to reflect real network behavior over time and remain resilient to emerging attack strategies. Detection confidence and feature-distribution statistics are also monitored to support future integration of automated drift detection.

The implemented code for this framework (model preprocessing and training) can be found at this GitHub URL: https://github.com/Rrezeartaa/DDoS_Prediction_with_ML_ThrtInt/ and the other part (the prediction part) is hosted in our EC2 servers.

Results and discussions

Our framework demonstrated a high level of effectiveness in real-time DDoS detection, maintaining near-perfect accuracy even under heavy network load. During controlled attack simulations, the system sustained log ingestion rates of over 10,000 events per second without any degradation in processing throughput or alerting latency. To evaluate the inference latency, we benchmarked the detection latency under production traffic load. Experiments were executed on an AWS t3.large instance (2 vCPUs @ 3.1 GHz, 8 GB RAM) hosting the prediction daemon. With a sustained log ingestion rate of 10-12k events/s and batch size of 64, the full inference pipeline achieved a mean latency of 34.2 ms and a 95th percentile latency of 47.6 ms. These results confirm the system satisfies the sub-50 ms real-time detection objective.

The trained XGBoost model proved to be extremely accurate: it achieved 99.999% accuracy on the test set (with precision and recall both at 1.0), and in live tests it continued to flag malicious traffic with virtually zero false positives.

As summarized in Table 3, all three classical machine learning models we evaluated (SVM, Random Forest, and XGBoost) and the deep learning model yielded excellent results on the combined dataset, but XGBoost slightly edged out the others in accuracy.

Figure 9 ilustrates the confusion matrices of the four evaluated models (SVM, Random Forest, XGBoost, and Deep Learning). Each classifier performed in distinguishing benign from DDoS traffic, with the near-perfect diagonal dominance confirming minimal misclassifications and reinforcing the overall reliability of the framework.

In addition to the comparative metrics summarized in Table 3, we also examined the learning dynamics of the deep learning baseline. Figure 10 ilustrates the training and validation accuracy and loss curves across epochs, confirming that the model converged quickly and maintained stable validation performance without overfitting.

For the final deployment, XGBoost was selected due to its marginal performance advantage and faster inference speed. The framework achieved approximately 99.99% accuracy with sub-50 ms inference latency on live pfSense traffic. This high accuracy stems from the enriched feature set—augmented by threat-intelligence indicators—and the integration of multiple heterogeneous datasets, which together enhance generalization. In operational terms, the detector successfully identified nearly all DDoS attack instances while maintaining negligible false positives on benign traffic, a crucial requirement for production environments where excessive alerts can lead to operator fatigue.

To contextualize these results, we compared our framework's performance with representative baselines reported in the literature. Prior works employing similar CIC-based datasets have demonstrated strong but slightly lower performance levels. Mishra et al. [27] achieved 99.76% accuracy with a Random Forest classifier on cloud-based traffic, AlSaleh et al. [30] reported 99.79% using a Bayesian CNN, and Sumukh and Sandhya [24] achieved 99.60% with an LSTM model for time-dependent anomalies. More recent hybrid approaches, such as the EDAD ensemble proposed by Abiramasundari and Ramaswamy [35], reached approximately 99% accuracy on CIC-DDoS2019.

Table 4 summarizes the comparative results. As shown, under comparable conditions our framework slightly exceeds reported results of the top-performing models by the literature by 0.2–0.3%, while achieving real-time inference speeds suitable for operational DDoS mitigation in cloud environments.

These comparisons illustrate that our framework not only matches but exceeds the accuracy of the best-performing methods in the literature review, while providing substantially lower latency and integrated consensus-based resilience, a dimension rarely explored in prior DDoS mitigation research.

We validated our framework under high-load conditions of a busy cloud environment. The ability to reliably process 15,000 log events per second with only minimal accuracy drop. For this experiment we used our best performed machine learning model. Table 5 shows accuracy

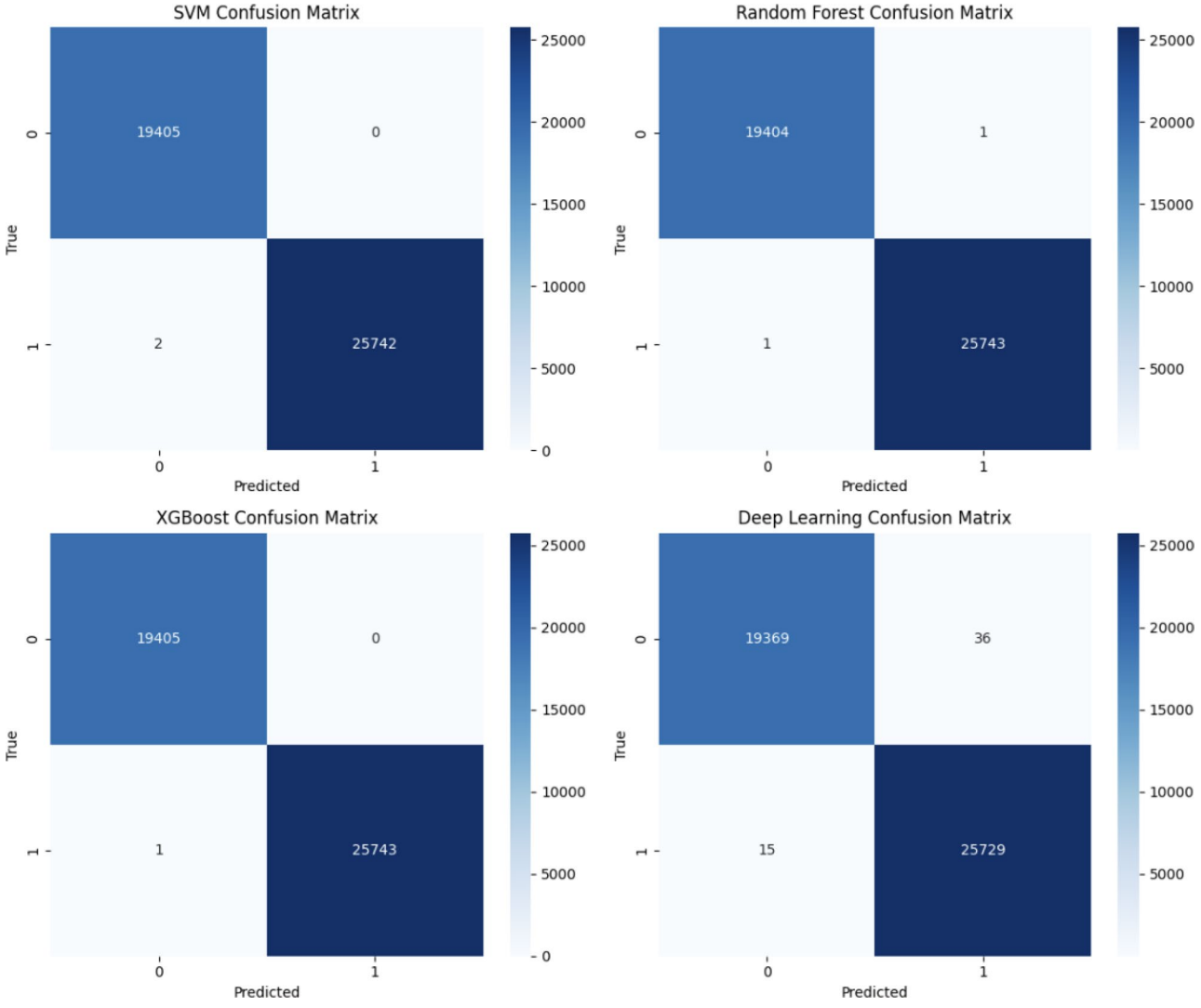


Fig. 9 Confusion matrices of the compared machine learning and deep learning models

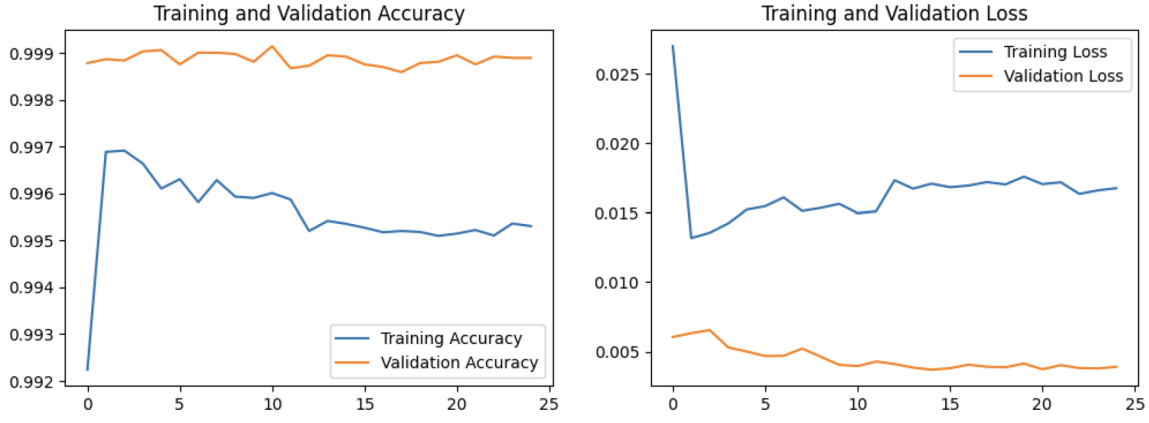


Fig. 10 Training and validation accuracy (left) and loss (right) curves for the deep learning baseline. The stable convergence and low validation loss demonstrate that the model generalized effectively to unseen data, supporting the high accuracy scores reported in Table 3

Table 4 Comparison with related work on CIC-based DDoS detection

Study	Method	Dataset	Accuracy (%)
[27]	Random Forest	CIC-IDS2017	99.76
[30]	Bayesian CNN	CIC-DDoS2019	99.79
[24]	LSTM	CSE-CIC-IDS2018	99.60
[35]	EDAD (PCA+RF/SVM)	CIC-DDoS2019	99.00
Our work	XGBoost	CIC-IDS2017 / CIC-DDoS2019 / pfSense + MISP Enrichment	**99.99**

Table 5 Xgboost model performance under varying traffic conditions

Traffic Intensity	Accuracy	Precision	Recall
Low (1,000 events/sec)	0.99998	1.00	1.00
Moderate (5,000 events/sec)	0.99997	1.00	1.00
High (10,000 events/sec)	0.99995	0.9999	1.00
Extreme (15,000+ events/sec)	0.99992	0.9998	0.9998

remained above 99.9% even at extreme volumes that indicates strong scalability.

In real deployments, a detector must maintain performance under traffic surges. Our pipeline's use of streaming ingestion and in-memory feature aggregation proved effective in this regard. By avoiding expensive operations (like disk I/O or heavy database transactions), we kept the detection loop lightweight. This design principle aligns with best practices in modern cloud-native security systems, which emphasize asynchronous, distributed processing of events.

One of the distinguishing features of our framework is the use of a consensus-like approach for alert validation across multiple replicas. Each replica server routes traffic through the pfSense gateway and contributes to the detection process by forwarding summary statistics. A quorum-based voting mechanism ensures that an impending attack prediction must be corroborated by a majority of monitoring nodes before automated mitigation is triggered. This design proved valuable in preventing false triggers: during testing, we intentionally introduced a faulty replica that output bogus alerts (simulating a compromised or mis-calibrated detector). The consensus logic successfully ignored the false alerts because the other replicas did not validate the same threat. Only genuine attacks, observed consistently by most monitors, resulted in system-wide action. This behavior is inspired by Byzantine Fault Tolerance, so as long as a majority of replicas are honest and observe the same traffic, the system reaches the correct decision. In addition, the replica setup also increases availability: replicas are distributed across regions and synchronize their state, so if one region is degraded or taken offline by a DDoS, unaffected replicas seamlessly maintain detection and response continuity. We validated this in practice by simulating a regional outage during an active DDoS attack simulation; the system continued to operate correctly as the remaining replicas preserved both monitoring and alerting functions.

The alerting subsystem, integrated with Wazuh and OpenSearch, consistently triggered actionable security alerts within milliseconds of a threat detection event, effectively mitigating potential DDoS impacts. Automated firewall rule updates executed via AWS Lambda functions demonstrated rapid response capabilities, typically within 120 milliseconds from detection to enforcement. We got 120 milliseconds by measuring the time from prediction on the Wazuh analytics node until enforcement of a filtering rule on pfSense.

- Model inference and alert generation: < 50 ms
- Wazuh → Lambda trigger: ≈ 30 ms
- pfSense rule update + propagation: 60–70 ms

Total reaction time: 120–140 ms. This confirms that the system can automatically block malicious traffic before a volumetric attack saturates network resources. Figure 11 ilustrates the alerts triggered in Wazuh after a DDoS attack was detected at one of the replicas.

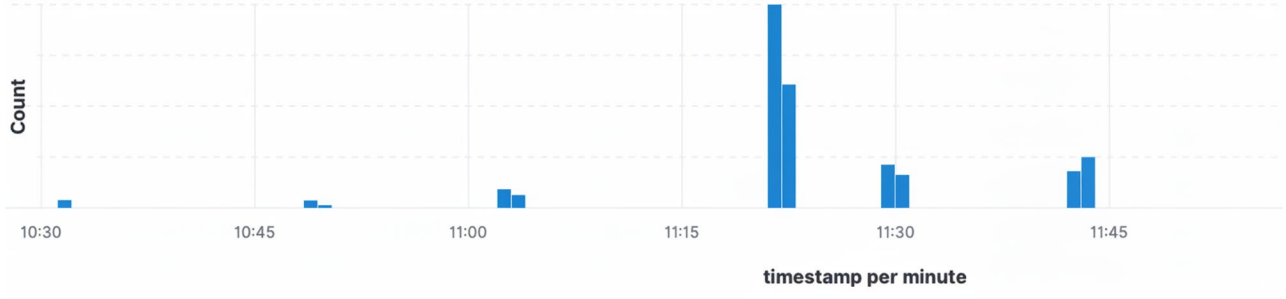


Fig. 11 Wazuh alerts from the DDoS attacks

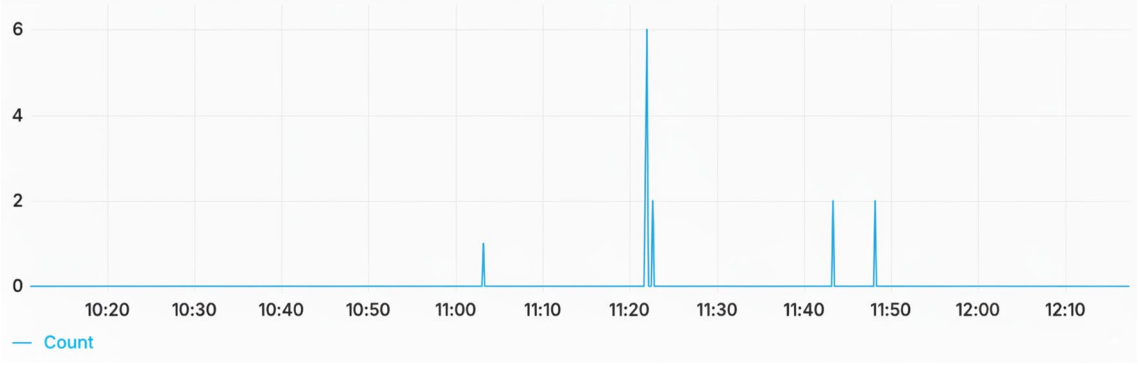


Fig. 12 Grafana dashboard from the alerts collected

Grafana visualization dashboards provided operators with comprehensive real-time insights, facilitating immediate threat recognition and proactive security management. Operators were able to quickly interpret visual cues from gauges, heat maps, and time series graphs to swiftly confirm and respond to detected threats. Figure 12 ilustrates the visualization done in the Grafana dashboards.

The results of our evaluation underscore a few important themes for cloud security. First, it is indeed possible to achieve both high accuracy and high performance in DDoS detection using a thoughtfully engineered ML pipeline. The often-cited trade-off between detection efficacy and system overhead can be mitigated by using efficient algorithms (like gradient-boosted trees) and streaming data handling, as we demonstrated. Second, the inclusion of threat intelligence data proved beneficial not just in theory but also in practice. Third, byzantine fault tolerance concepts, when pragmatically applied, can elevate the trustworthiness of a security system. In our case, even if one component malfunctioned or was compromised, the system as a whole maintained correct operation, which is crucial for dependable defenses. We measured consensus failures and false-positive activations across 100 adversarial rounds. Even with one Byzantine node, the system maintained correct operation and no false enforced rules were observed.

Exploratory LLM-based testings are omitted from the main evaluation and summarized separately in Appendix A, as they do not represent operational baselines for real-time DDoS detection.

The evaluation relies on a combination of public benchmark datasets and proprietary pfSense traffic captures. Although temporal partitioning and replay controls were applied to reduce data leakage, these datasets may still introduce bias toward known traffic patterns and attack behaviors, potentially inflating performance compared to fully unseen real-world deployments. Furthermore, the evaluation focuses exclusively on supervised learning approaches; unsupervised and semi-supervised baselines were not included. While such techniques are relevant for zero-day discovery, they are generally less suited for precise, low-latency operational alerting and remain an important direction for future work.

Conclusion and future work

This paper introduced a proactive real-time framework for DDoS detection and mitigation in cloud environments, combining machine learning, threat intelligence, and lightweight consensus mechanisms. By integrating pfSense for traffic monitoring, Wazuh for analytics, and XGBoost for prediction, the system achieved near-perfect accuracy (≈99.99%), sub-50 ms inference latency, and scalability beyond 15,000 events/sec. These results confirm that carefully engineered ML pipelines, enriched with contextual threat intelligence, can overcome the high false-positive rates and rigidity of traditional IDS systems.

Beyond detection accuracy, a key contribution lies in the system's resilience. Inspired by Byzantine Fault Tolerance principles, replicated monitoring nodes reached quorum before enacting mitigation, ensuring consistency even in the presence of faulty or compromised components. This consensus-lite design, validated through simulated outages and false alerts, underscores the importance of reliability and trustworthiness in operational defenses.

Based on the exploratory experiments, the comparative evaluation with generative AI models (OpenAI, DeepSeek, and Grok) suggests limitations of LLM-based approaches when applied to high-frequency, structured network telemetry. While such models may be useful for retrospective analysis and higher-level threat interpretation, they did not consistently meet the determinism and low-latency requirements necessary for inline predictive enforcement in our setting.

Looking ahead, several directions remain open. First, integrating automated enforcement, e.g., dynamic firewall rule orchestration via pfSense APIs will enable end-to-end closed-loop defense. Second, future studies

should incorporate unsupervised and semi-supervised baselines to complement supervised detection, particularly for identifying previously unseen or evolving attack patterns. Third, applying this framework to consensus-driven systems such as blockchains or replicated services represents a promising but underexplored research direction, especially in combining protocol-layer telemetry with ML-based prediction. Addressing these challenges will help advance the state of proactive, resilient cloud security.

Appendix A: exploratory LLM-based experiments

This appendix documents exploratory experiments conducted with large language models (GPT, DeepSeek, and Grok) to assess their applicability to structured, numeric flow-level telemetry in the context of DDoS detection. These experiments are included for completeness and to explore emerging research directions; they are not part of the core methodology, were not used for model selection or deployment decisions, and are not intended as competitive baselines against the structured machine learning models evaluated in the main body of the paper.

To enable this exploratory assessment, flow-level network statistics were reformulated into a textual representation compatible with prompt-based inference. Specifically, each flow record was serialized into a structured prompt string, for example: "Flow ID: XYZ; Packet Count: 120; BPS: 45,000", together with additional attributes such as flow byte rate, TCP flag distribution, and entropy estimates. Listing 7 illustrates an excerpt of the prompt construction and inference pipeline used to feed flow-level statistics into the generative AI models.

Listing 7 Excerpt of the exploratory prompt-based inference process for generative AI models (OpenAI example)

```
prompt = f"""
Flow Statistics:
- Byte rate: {flow_features['byte_rate']}
- Packet count: {flow_features['packet_count']}
- TCP flags: {flow_features['tcp_flags']}
- Entropy: {flow_features['entropy']}
Classify whether this flow is benign or
part of a potential DDoS attack.
"""

# Send the prompt to the OpenAI model
response = gpt_model.generate(prompt)
print("Prediction:", response)
```

Using this prompt-based formulation, we evaluated the behavior and operational characteristics of generative models under realistic traffic conditions. The following discussion summarizes the key limitations observed when applying this approach in a real-time DDoS detection setting.

While generative models offer substantial interpretability and contextual reasoning, their use in this real-time detection framework presented several limitations. Firstly, the input datasets were highly structured and numerical, which required significant preprocessing and prompt engineering to be compatible with the language-model-driven inference pipeline. This transformation not only introduced additional complexity but also increased the time needed to formulate valid queries and parse outputs.

Moreover, although models like Grok demonstrated relatively faster inference and were specifically optimized for anomaly detection domains, their latency still could not consistently compete with the sub-50 ms thresholds required for real-time packet-level decisioning. DeepSeek and GPT models, though accurate in their general assessments, suffered from token-processing delays and inconsistent outputs when parsing highly technical flow metrics.

These results confirm that LLMs trained for natural language processing are not well suited for direct classifica-

tion of numeric flow telemetry, which motivates our focus on specialized structured ML classifiers for operational deployment. High-frequency, multivariate numerical traffic proved suboptimal for language models. As a result, while the generative models show potential for retrospective analysis such as high-level summarization and trend interpretation, they lack the determinism and latency guarantees necessary for inline predictive enforcement in real-time DDoS mitigation.

While our experiments demonstrated that general-purpose LLMs such as GPT, DeepSeek, and Grok struggle with structured network telemetry, related work has explored adapting language models for anomaly detection in communication systems. For instance, FedLLMGuard [41] applies federated LLM architectures to 5G network anomaly detection, highlighting the feasibility of distributed, privacy-preserving inference. However, unlike our flow-level numeric inputs, FedLLMGuard operates on semi-structured textual logs, emphasizing the complementary but distinct scope of LLM-based approaches compared to our optimized structured-feature pipeline.

Acknowledgements
The authors express their sincere appreciation to Interdisciplinary Centre for Security, Reliability and Trust (SNT) of University of Luxembourg for supporting this research.

Author contributions
R.T.: Conceptualization, methodology design, system implementation, dataset integration, experiments, and manuscript drafting. G.L.: Review, validation of methodology, and critical manuscript revision. M.V.: Fault-tolerance and consensus model design, validation of resilience strategies, and manuscript editing. B.R.: Supervision, project administration, conceptualization, methodology design, guidance on research direction, and final approval of the manuscript. All authors have reviewed and approved the final version of the manuscript.

Funding
This research received no external funding.

Data availability
A part of the implemented code and data set used for this research is available at: https://github.com/Rrezeartaa/DDoS_Prediction_with_ML_ThrtInt/.

Declarations

Competing interests
The authors declare no competing interests.

Received: 9 September 2025 / Accepted: 16 January 2026
Published online: 04 February 2026

References

1. Cloudflare DDoS threat report for 2025 Q2. https://blog.cloudflare.com/ddos-threat-report-for-2025-q2/. Accessed Aug 22 2025
2. N V, Sriram VSS (2025) Mitigating DDoS attacks in virtual machine migration: an in-depth security framework utilizing deep learning and advanced encryption techniques. Int J Innovative Technol Exploring Eng. https://doi.org/10.35940/ijitee.B1032.14020125
3. Thaqi R, Krasniqi B, Mazrekaj A, Rexha B (2025) Literature review of machine learning and threat intelligence in cloud security. IEEE Access 13. https://doi.org/10.1109/ACCESS.2025.3529636
4. Alshammari A, Aldribi A (2021) Apply machine learning techniques to detect malicious network traffic in cloud computing. J Big Data 8(90). https://doi.org/10.1186/s40537-021-00475-1
5. Sharma A, Singh UK (2025) Cloud computing security through detection & mitigation of zero-day attack using machine learning techniques. Chap. 25:357–388. https://doi.org/10.1002/9781394272464.ch25
6. Rai HM, Yoo J, Agarwal S (2024) The improved network intrusion detection techniques using the feature engineering approach with boosting classifiers. Mathematics 12(24). https://doi.org/10.3390/math12243909
7. Joseph O, Ajax R (2025) Comparison of traditional vs. ai-based intrusion detection and prevention systems: efficiency and accuracy. J Cybersecur Adv
8. Mohale VZ (2025) Evaluating machine learning-based intrusion detection: improved accuracy, reduced false positives, and identification of unseen threats. Front Comput Sci. https://doi.org/10.3389/fcomp.2025.1520741. published online in 2025
9. Rahman MR, Mahdavi-Hezaveh R, Williams L (2021) What are the attackers doing now? automating cyber threat intelligence extraction from text on pace with the changing threat landscape: a survey. arXiv preprint arXiv:2109.06808 https://doi.org/10.48550/arxiv.2109.06808
10. Sousa P, Neves NF, Verissimo P, Sanders WH (2006) Proactive resilience revisited: the delicate balance between resisting intrusions and remaining available. In 2006 25th IEEE Symposium on Reliable Distributed Systems (SRDS'06), pp 71–82. https://doi.org/10.1109/SRDS.2006.37
11. Silva DS, Graczyk R, Decouchant J, Völp M, Esteves-Verissimo P (2021) Threat adaptive byzantine fault tolerant state-machine replication. In 2021 40th International Symposium on Reliable Distributed Systems (SRDS), pp 78–87. https://doi.org/10.1109/SRDS53918.2021.00017
12. Zhong W, Yang C, Liang W, Cai J, Chen L, Liao J, Xiong N (2023) Byzantine fault-tolerant consensus algorithms: a survey. Electronics 12(18):3801. https://doi.org/10.3390/electronics12183801
13. Chen S-R et al. (2025) Enhancing machine learning-based ddos detection with adaptive hyperparameter tuning for robust generalization. Electronics 14(16):3319. https://doi.org/10.3390/electronics14163319
14. Alrassan I, AlQahtani AM (2023) Detection of ddos attacks on clouds computing environments using machine learning techniques. In 2023 International Conference on Communication and Network Security (ICCNS). https://doi.org/10.1109/iccns58795.2023.10193141
15. Polu S, Bapuji V, Supervisor R (2024) Mitigating ddos attacks in cloud computing using machine learning. Braz J Devel. https://doi.org/10.34117/bjdv10n1-022
16. Mohd M, Ab MF, Azmi ZRBM, Firdaus A, Nuhu AH, Hussain SS (2024) Machine learning and deep learning approaches for detecting ddos attacks in cloud. Future Predictive Analytics. https://doi.org/10.54216/fpa.170207
17. Sathvika C, Satwika V, Sruthi Y, Geethika M, Bulla S, Swathi K (2023) Ddos attack detection on cloud computing services using algorithms of machine learning: survey. In 2023 7th International Conference on Computing Methodologies and Communication (ICCMC), pp 1094–1100. https://doi.org/10.1109/ICCMC56507.2023.10083549
18. Arunkumar R, Navanitha S, Padmavathi B, Snekaa V (2024) Hybrid svm approach for enhanced ddos attack detection using machine learning in cloud environment. In 2024 2nd International Conference on Artificial Intelligence and Machine Learning Applications Theme: Healthcare and Internet of Things (AIMLA), pp 1–4. https://doi.org/10.1109/AIMLA59606.2024.10531330
19. Reddy SPK, Nagavelli U, Kiran YS, Kondoju CS, Bushmoni S, Yashaswi A (2024) Deep learning for zero-day threat detection and mitigation. In 2024 International Conference on IoT Based Control Networks and Intelligent Systems (ICICNIS), pp 1362–1368. https://doi.org/10.1109/ICICNIS64247.2024.10823270
20. Benjamin M (2025) Real-time threat intelligence feeds and machine learning fusion: proactive security in dynamic network environments
21. Freitas T, Soares J, Correia ME, Martins R (2023) Deterministic or probabilistic? – a survey on byzantine fault tolerant state machine replication. Comput Secur 129:103200. https://doi.org/10.1016/j.cose.2023.103200
22. Castro M, Liskov B (1999) Practical byzantine fault tolerance. In Proceedings of the Third Symposium on Operating Systems Design and Implementation. OSDI'99, USENIX Association, USA, 173–186
23. Liu X, Zhang Z, Li Z, Lu X, Li M, Xu L, Ao M, Zhu L (2025) Group bft: two-round bft protocols via replica grouping. IEEE Trans Dependable Secure Comput. https://doi.org/10.1109/TDSC.2025.3583633

24. Sumukh S, Sandhya S (2024) Ddos detection using ml and deep learning approaches. In CSITSS 2024. https://doi.org/10.1109/csitss64042.2024.10817014
25. Subrmanian K, Thangarasu G, Yanyan Z, Kannan KN (2024) Enhancing detection and prediction of ddos attacks through regression modeling. In 2024 IEEE 6th Symposium on Computers & Informatics (ISCI), pp 253–257. https://doi.org/10.1109/ISCI62787.2024.10668039
26. Makkawi AM, Yousif A (2021) Machine learning for cloud ddos attack detection: a systematic review. In 2020 International Conference on Computer, Control, Electrical, and Electronics Engineering (ICCCEEE), pp 1–9. https://doi.org/10.1109/ICCCEEE49695.2021.9429678
27. Mishra A, Gupta BB, Peraković D, Peñalvo FJG, Hsu C-H (2021) Classification based machine learning for detection of ddos attack in cloud computing. In 2021 IEEE International Conference on Consumer Electronics (ICCE), pp 1–4. https://doi.org/10.1109/ICCE50685.2021.9427665
28. Aliar AAS, Agoramoorthy M, Justindhas Y (2023) An automated detection of ddos attack in cloud using optimized weighted fused features and hybrid dbn-gru architecture. Cybern Syst 55(7):1469–1510. https://doi.org/10.1080/01969722.2022.2157603
29. Manimaran A, GnanaJeyaraman R, J Carmel Mary Belinda M, M S, Kannan E, Sivaram M (2024) An adaptive framework for low-rate ddos detection in cloud environments using decision tree machine learning algorithm. In 2024 IEEE International Conference on Blockchain and Distributed Systems Security (ICBDS), pp 1–5. https://doi.org/10.1109/ICBDS61829.2024.10837242
30. AlSaleh I, Al-Samawi A, Nissirat L (2024) Novel machine learning approach for ddos cloud detection: Bayesian-based cnn and data fusion enhancements. Sensors. https://doi.org/10.3390/s24051418
31. Green SB (2021) Advancements in cloud security: leveraging machine learning for threat detection. Univers Res Rep. https://doi.org/10.36676/urr.v8.i4.1400
32. Corrêa JHGM, Ciarelli PM, Ribeiro MRN, Silva Villaça R (2021) Ml-based ddos detection and identification using native cloud telemetry macroscopic monitoring. J Network Syst Manag. https://doi.org/10.1007/S10922-020-09578-1
33. Krishna DV, Sneha E, Latha, Yadav B, Aswini J, Harshini K (2024) Securing the cloud: a machine learning approach for threat detection and mitigation. Int J Adv Res Sci Commun Technol (IJARSCT). https://doi.org/10.48175/ijarsct-18812
34. Chauhan S, Byahatti P, Patel SK (2024) Securing networks: leveraging machine learning for enhanced ddos detection. In 2024 First International Conference on Software, Systems and Information Technology (SSITCON), pp 1–8. https://doi.org/10.1109/SSITCON62437.2024.10796323
35. Abiramasundari S, Ramaswamy V (2025) Distributed denial-of-service (ddos) attack detection using supervised machine learning algorithms. Sci Rep 15(1):5678. https://doi.org/10.1038/s41598-024-84879-y
36. Feng H, Lu Z, Mai T, Tang Q (2025) Faster hash-based multi-valued validated asynchronous byzantine agreement. In Proceedings of the 55th IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), pp 303–316. https://doi.org/10.1109/DSN64029.2025.00040
37. Wu C, Qin H, Amiri MJ, Loo BT, Malkhi D, Marcus R (2025) Bftbrain: adaptive BFT consensus with reinforcement learning. In Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI). https://doi.org/10.48550/arXiv.2408.06432
38. Fu X, Li M, Zeng Q, Li T, Yang S, Guan Y, Liu C (2025) Hamster: a fast synchronous byzantine fault tolerant protocol. IEEE Trans Inf Forensics Secur 20(12):2664–2676. https://doi.org/10.1109/TIFS.2025.3544034
39. Songa AV, Karri GR (2024) An integrated sdn framework for early detection of ddos attacks in cloud computing. J Cloud Comput 13:64. https://doi.org/10.1186/s13677-024-00625-9
40. Li Y, Zhang H, Xu K (2023) Model poisoning attack against federated learning with adaptive aggregation. IEEE Trans Inf Forensics Secur 18:5456–5468. https://doi.org/10.1109/TIFS.2023.3345012
41. Wang X, Zhao Y, Chen L, Li J (2025) FedIlmguard: a federated large language model for anomaly detection in 5g networks. Comput Networks. https://doi.org/10.1016/j.comnet.2025.110021

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.